AKSUM UNIVERSITY
INSTITUTE OF COMPUTING TECHNOLOGY
DEPARTMENT: COMPUTER SCIENCE

4/1/2026 fix

Project report of greedy optimization: cargo loading for shipping company

Group Members

- Dagmawit meresa ---------- aku1603899
- Samuel ataklti ----------aku1603833
- Betelihem haile---------- aku1603359

Table of Contents

1. Introduction 1
2. Algorithm Design 1
3. Implementation 2
4. Complexity Analysis 3
Conlcusion 6
Challenges Faced 7
References 7

1. Introduction

The **Knapsack Problem** is an optimization problem where items with given weights and profits are selected to maximize total profit in a knapsack of limited capacity. There are two algorithms used to solve knapsack problems mainly the fractional knapsack and 0/1 knapsack dynamic programming

industries for different logistics such as cargo shipping in rail ways , and airport industries. to optimize profit using set of steps.

In data structure there are different greedy algorithms for profit optimization in different industries specifically in the Shipping

Based on the above introduction concepts of the two algorithms used here we can just formulate the problem of statement as shown below:

Given a set of n items, each with a profit p_i and weight w_i, the objective is to maximize total profit under a capacity constraint C:

$$\text{maximize } P = \sum_{i=1}^{n} p_i x_i \text{ subject to } \sum_{i=1}^{n} w_i x_i \leq C$$

- For Fractional Knapsack, $0 \leq x_i \leq 1$
- For 0/1 Knapsack, $x_i \in \{0,1\}$

2. Algorithm Design

Fractional Knapsack (Greedy Approach):

Idea:
Select items based on the highest profit-to-weight ratio $\rho_i = p_i / w_i$ until capacity is full.

Pseudocode:

```
Sort items by profit-to-weight ratio in descending order
Initialize total_profit = 0, remaining_capacity = C
for item in sorted_items:
    if item.weight <= remaining_capacity:
        total_profit += item.profit
        remaining_capacity -= item.weight
```

```
    else:
        total_profit += item.profit * (remaining_capacity / item.weight)
        break
return total_profit
```

0/1 Knapsack (Dynamic Programming)

Basic Idea:
Build a DP table dp[i][w] storing the maximum profit using the first i items with capacity w.

Pseudocode:

```
Initialize dp[0..n][0..C] = 0
for i = 1 to n:
    for w = 0 to C:
        if item[i].weight <= w:
            dp[i][w] = max(dp[i-1][w], dp[i-1][w-item[i].weight] + item[i].profit)
        else:
            dp[i][w] = dp[i-1][w]
return dp[n][C]
```

Why Fractional knapsack(greedy algorithm)

The greedy approach is used because the problem allows to the fractions of next high density of item or profit to weight ratio, and it ensures that every unit of the capacity yields the maximum profit gain. As the dataset increases the profit will also increases as remaining amount or fraction of items will be included.

Why 0/1 knapsack

In 0/1 Knapsack, you cannot take fractions. A high-ratio item might leave a "gap" in the knapsack that is too small for other items but large enough that a combination of lower-ratio items would have been more profitable. Fractional knapsack eliminates this issue by allowing the "gap" to be filled exactly.

3. Implementation

For implementing the above problem we used a python programming language implementing the two algorithms used such as Fractional knapsack and 0/1 knapsack or dynamic programming. For example The data structure contains 1000 random generated with the following data members Id, weight , Profit, and Ratio. For implementing the two algorithms

we used random library to generate random dataset, time to capture the start and end time to execute the algorithms, and csv for export the knapsack values.

Functions used in the implementation includes:

fractional_knapsack used to solve the cargo shipping problem using greedy or fractional knapsack based on the two parameters item and capacity

Zero_one_knapsack is used to solve the above problem based on dynamic programming and items and capacity parameters used to execute the function.

Export Fractional knapsack: This function used to export the knapsack fractional solution or results to csv files and latter it will easy for analysis the data output, and detail complexities.

Export zero_one_ Solution: This function used to export the 0/1 knapsack solution or results to csv files and latter it will easy for analysis the data output, and detail complexities.

Export_to_csv: this function will be used to export all the initial 1000 dataset generated by the function.

Arrays Used:

Item: to store the list of random generated items in the array as it will be used for fractional knapsack and 0/1 knapsack.

Capacity:

Dp: is used to create the 2D Array for the items and weight and the table is created by item+1 * weight +1. This will be used in the 0/1 knapsack dynamic programming algorithm.

4. Complexity Analysis

Complexity Analysis of Fractional knapsack

Time complexity

The steps to execute the function using this algorithm will pass the following steps:

Step1: Compute profit/weight ratio for all data items this will have O(n) time complexity.

Step 2: Sort items ratio descending this will have two loops one to iterate or the list of items and the other sorting merge sort and will have O(nlogn)

Step3: Iterate items and take until the capcity by considering the fraction values of the next high density or profit/ weight ratio. This will take O(n)

When we sum the three steps time complexity as in the shwon below:

Total Time complexity=O(n)+ O(nlogn)+O(n)=O(nlogn) assu,img that we ignored the O(1) for assignmnets.

Space Complexity:

To calculate the space complexity of the fractional knapsack algorithm it stores the n number of items in the array so this will have O(n) space to handle the dat**a.**

0/1 Knapsack

In this algorithm there two for lops used one the outer loop to iterate the list of items from the array, and the second inner loop iterate the weight or capacity of the items. So the time complexity will be **O(n*W**) where n is the number of items and w is the number of weight or capacity given in the dataset.

Space Complexity:

This algorithm uses DP 2D Array of n+1*c+1 where n is the numer if items and c is the capacity. Each cell contains an integer values and space or memory of **O(n*C)**

Expermentation

```
C:\Users\O'TECH\Desktop\Algorithm Analysis>py solution.py
Dataset exported to knapsack_dataset.csv
Capacity %   | Total Profit    | Execution Time
-----------------------------------------------
Fractional solution exported to fractional_50.csv
      50.0% |       203878.18 | 0.013574s
Fractional solution exported to fractional_75.csv
      75.0% |       242464.80 | 0.009127s
Fractional solution exported to fractional_90.csv
      90.0% |       253276.67 | 0.007685s
Dataset exported to small_dataset.csv
Fractional solution exported to small_fractional_solution.csv
0/1 solution exported to small_zero_one_solution.csv

--- Verification ---
Small Dataset (n=10, Cap=15)
Fractional (Greedy) Profit: 287.14
0/1 (DP) Profit: 263
Note: Fractional >= 0/1 because fractions are allowed.
```

From the above screenshot from the experment it clearly shows that multiple capacities test cases was used to exceute the algorothms to get the maximum profit. Test cases 50% , 75%, and 90% were used to check the profit and time exceution for the algorithmd used,

From the random generated data of using 50% capacity from 1000 random generated data the maxi,u, profit gained is 203878.18 unit and exceution time 0.013574, 75% capacity of from the dataset 242464.8, and time exceution 0.009127, and 90% 2523276.67 profit and time of 0.007685. For verification purpose n =10 was used and capacity of 15 was tested and for fractional or greedy knapsack 287.14 profit gained and for the 0/1 dynaimic programming is 263. this cleraly indicates profit usnig fractional or greedy knapsack is always greater or equal to the 0/1 knapsack.

Sample 10 tasks data for verification:

```
ID,Weight,Profit,Ratio
0,7,80,11.43
1,8,10,1.25
2,4,79,19.75
3,10,32,3.2
4,6,32,5.33
5,8,67,8.38
6,3,62,20.67
7,4,33,8.25
8,3,89,29.67
9,10,67,6.7
```

From the sample 10 tasks dataset and capacity of 15. Using the greedy or fractional the following result is generated.

```
Item ID,Fraction Taken
8,1.0
6,1.0
2,1.0
0,0.71
```

Sample dataset for verification usimg 0/1 knapsack

```
Item ID,Weight,Profit
2,4,79
6,3,62
7,4,33
8,3,89
```

Conlcusion

What I Learned

Through this assignment, I developed a clear understanding of how optimization problems like the Knapsack Problem can be solved using different algorithmic approaches. I learned that the **Fractional Knapsack** uses a greedy strategy based on the profit-to-weight ratio, which ensures that each unit of capacity is used efficiently to maximize profit. This helped me understand how greedy algorithms make locally optimal choices that lead to a globally optimal solution when the problem allows fractional selection. I also learned how sorting plays a key role in improving efficiency, leading to a time complexity of O(nlog n)O(n \log n)O(nlogn), which is suitable for large datasets.

In addition, I gained practical knowledge of the **0/1 Knapsack problem** using dynamic programming. I understood how to construct and use a 2D DP table to store intermediate results and make optimal decisions for each item and capacity. This approach helped me see

how problems with overlapping subproblems and optimal substructure can be solved efficiently. I also learned how time and space complexity differ between greedy and dynamic programming approaches, and why dynamic programming is necessary when items cannot be divided. Implementing both algorithms in Python improved my programming skills, especially in handling arrays, loops, and exporting results for analysis.

Challenges Faced

One of the main challenges I faced was understanding and implementing the dynamic programming approach for the 0/1 Knapsack problem, particularly how the DP table is built and updated step by step. It was also difficult to manage large datasets (like 1000 items) and ensure the correctness of results while measuring execution time. Additionally, comparing the performance of both algorithms and interpreting the results required careful analysis. Debugging the code and ensuring proper data export to CSV files also took time, but these challenges helped me improve my problem-solving and coding skills.

References

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

[2] A. V. Aho, J. E. Hopcroft, and J. D. Ullman, *Data Structures and Algorithms*. Reading, MA, USA: Addison-Wesley, 1983.

[3] D. E. Knuth, *The Art of Computer Programming, Vol. 1: Fundamental Algorithms*, 3rd ed. Reading, MA, USA: Addison-Wesley, 1997.

[4] S. Dasgupta, C. H. Papadimitriou, and U. V. Vazirani, *Algorithms*. New York, NY, USA: McGraw-Hill, 2008.